



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERIA
Escuela Profesional de Ingeniería de Sistemas

**INFORME DE LABORATORIO “Ejercicio
LINQ”**

Curso: Programación II

Docente: BRENO SOLIM LUQUE ZÚÑIGA

- **Jiménez Romero, Josué Andre (2022074259)**

Tacna – Perú
2026



INDICE

Contenido

INDICE	2
I. INFORMACIÓN GENERAL	3
- Objetivos	3
- Equipos, materiales, programas y recursos utilizados	3
II. MARCO TEORICO	3
III. PROCEDIMIENTO	4
IV. ANALISIS E INTERPRETACION DE RESULTADOS	14
CONCLUSIONES	14
RECOMENDACIONES	14



INFORME DE LABORATORIO

TEMA: Ejercicio LINQ

I. INFORMACIÓN GENERAL

- **Objetivos:** Desarrollar una aplicación de escritorio utilizando C# y Windows Forms, aplicando buenas prácticas de programación y el uso de consultas LINQ para la manipulación, filtrado y cálculo sobre colecciones de datos.
- **Equipos, materiales, programas y recursos utilizados:**
 - IDE de desarrollo (Visual Studio 2022 o similar).
 - Lenguaje de programación C# (.NET Framework / .NET Core).
 - Computadora con sistema operativo Windows y al menos 4GB de RAM.

II. MARCO TEORICO

El desarrollo de la práctica se enfocó en la creación de interfaces gráficas y la lógica algorítmica aplicada a diferentes casos de uso:

- **Parte I: Tecnología LINQ:**

Herramienta de Microsoft introducida en .NET Framework 3.5 que permite realizar consultas de manera uniforme sobre diferentes fuentes de datos. LINQ unifica las operaciones comunes en un único lenguaje, simplificando el acceso y manipulación de datos. Para este proyecto, se utilizó la sintaxis de métodos basada en expresiones lambda.

- **Parte II: Componente DataGridView:**

Elemento fundamental utilizado para mostrar la lista de personas registradas, permitiendo visualizar los datos tabulados (Nombre y Edad) de forma clara y



actualizarse dinámicamente tras aplicar filtros de búsqueda.

- **Parte III: Colecciones Genéricas (List) :**

Se utilizó una lista global genérica (**List<Persona>**) para almacenar los objetos creados en memoria. Esta colección sirvió como la fuente de datos principal sobre la cual iteraron las funciones de LINQ.

- **Parte IV: Controles de Interfaz y Eventos :**

Se aprovechó el entorno de Windows Forms utilizando eventos como **Click** (para agregar, filtrar por edad y promediar) y el evento dinámico **TextChanged** (para filtrar en tiempo real conforme el usuario escribe en la barra de búsqueda).

III. PROCEDIMIENTO

Figura 1

Diseño de formulario MENU

Nombre	Edad
Luis	23
Nestor	21
Diego	24
Jefferson	18
Aldair	19
Pepe	19

Figura 2



Boton Agregar

```
1 referencia
private void btnAgregar_Click(object sender, EventArgs e)
{
    if (string.IsNullOrEmpty(txtNombre.Text) || string.IsNullOrEmpty(txtEdad.Text))
    {
        MessageBox.Show("Por favor, ingresa el nombre y la edad.", "Campos vacíos", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    if (int.TryParse(txtEdad.Text, out int edad))
    {
        Persona nuevaPersona = new Persona
        {
            Nombre = txtNombre.Text,
            Edad = edad
        };

        listaPersonas.Add(nuevaPersona);

        ActualizarGrid(listaPersonas);

        txtNombre.Clear();
        txtEdad.Clear();
        txtNombre.Focus();
    }
    else
    {
        MessageBox.Show("Ingresa un número válido para la edad.", "Error de formato", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}
```

Nota: Se establecen validaciones, como no dejar campos vacíos, verificación de formato entre otros

Figura 3

Función Actualizar GRID

```
3 referencias
private void ActualizarGrid(List<Persona> lista)
{
    dataGridView1.DataSource = null;
    dataGridView1.DataSource = lista;
}
```

Nota: Primero se limpia y luego se carga la información de Lista

Figura 4



Boton “Filtrar mayores de edad”

```
1 referencia
private void btnMayores_Click(object sender, EventArgs e)
{
    var mayoresDeEdad = listaPersonas.Where(p => p.Edad >= 18).ToList();

    ActualizarGrid(mayoresDeEdad);
}
```

Figura 5

Boton “Calculo de promedio de edades”

```
1 referencia
private void btnPromedio_Click(object sender, EventArgs e)
{
    if (listaPersonas.Count > 0)
    {
        double promedio = listaPersonas.Average(p => p.Edad);

        lblResultado.Text = $"Promedio: {promedio:F1}";
    }
    else
    {
        lblResultado.Text = "Promedio: --";
        MessageBox.Show("No hay personas agregadas para calcular el promedio.", "Lista vacia", MessageBoxButtons.OK, MessageBoxIcon.Information);
    }
}
```

Figura 6

Boton “Calculo de promedio de edades”

```
1 referencia
private void txtBuscar_TextChanged(object sender, EventArgs e)
{
    string filtro = txtBuscar.Text.ToLower();

    var personasFiltradas = listaPersonas
        .Where(p => p.Nombre.ToLower().Contains(filtro))
        .ToList();

    ActualizarGrid(personasFiltradas);
}
```

IV. ANALISIS E INTERPRETACION DE RESULTADOS

El desarrollo del ejercicio permitió evidenciar la versatilidad y eficiencia de C# al trabajar con colecciones. Las validaciones iniciales demostraron ser



efectivas al impedir el registro de campos vacíos o la introducción de texto en el campo de edad, asegurando la integridad de los datos. La implementación de LINQ simplificó drásticamente la lógica algorítmica: el uso del método **Where** facilitó la extracción inmediata de las personas mayores de edad y permitió implementar una búsqueda dinámica (en el evento **TextChanged**) sin necesidad de recurrir a bucles **for** o **foreach** complejos y anidados. Por otro lado, la integración del **DataGridView** hizo muy visual el cambio de estado de la información, mientras que el uso de la función matemática **.Average()** de LINQ permitió calcular el promedio de edades con una sola línea de código, validando previamente que la lista no estuviera vacía para evitar errores de división por cero.

V. CONCLUSIONES

- El uso de la herramienta LINQ reduce significativamente la cantidad de código necesario para buscar, filtrar y realizar cálculos matemáticos sobre listas, mejorando la legibilidad, la limpieza y el mantenimiento del proyecto en comparación con las sentencias condicionales tradicionales.
- El diseño de interfaces va más allá de tener una buena apariencia estética; implica el uso estratégico de validaciones y controles para evitar excepciones en tiempo de ejecución (como intentar calcular un promedio sin datos o ingresar letras donde se esperan números).

VI. RECOMENDACIONES

- Implementar métodos de visualización (como un botón "Mostrar Todos" o "Limpiar") en el formulario para restablecer rápidamente la grilla de datos a su estado original, facilitando la realización de múltiples pruebas sin tener que borrar manualmente el texto de la barra de búsqueda.
- Modularizar las clases. Aunque para este ejercicio la clase **Persona** convive



en el mismo archivo del formulario, para proyectos más grandes se recomienda separar los modelos de datos en sus propios archivos (.cs) o carpetas, mejorando la arquitectura del software.